

Building a Training Set for Drone-and-Buggy Litter Detection

Gap analysis of the current image set and a capture plan

August 5, 2026

Summary

The target system is a three-stage pipeline: a 4K drone scans from high altitude, descends for a low-altitude pass, and a ground buggy with a camera and arm retrieves the objects. Each stage sees objects at a completely different scale, from a completely different angle, and the current image set supports *none of them*.

Three findings drive everything below.

1. **There is no aerial imagery at all.** All 244 images are handheld iPhone photographs taken from roughly waist height. The directory scaffold for aerial, occlusion, lighting and weather variation exists but contains only 27 `.gitkeep` placeholder files and zero images.
2. **Detecting cigarette butts from 20 m is not possible with the Neo 2.** This is a physical limit, not a model-quality problem (Section 2). The Neo 2's 16.5 mm-equivalent lens is very wide, so at 20 m a butt spans about *two* pixels, and even a plastic bottle is marginal. Butts require flying at 1.5–2 m; the survey pass should drop from 20 m to 10–12 m. No amount of training changes these numbers.
3. **The set is too small and too unbalanced to train on.** After classification, 107 of 244 images contain no identifiable object, and 13 of the 23 classes have five examples or fewer. Useful object detection needs on the order of 10^3 instances per class, not 10^0 .

The good news is that the hard part — knowing what the target objects look like in situ, and a working manual polygon annotation workflow — already exists and should be preserved.

1 What the dataset currently is

Property	Value
Images	244
Capture device	iPhone SE (3rd gen), 242/244
Resolution	4032 × 3024 (12 MP), 240/244
Images with GPS EXIF	242
Viewpoint	Handheld, ground level, near-nadir
Altitudes represented	One (≈1.2 m handheld)
Manually annotated files	100
Manual polygons	331
— of which are background (rock, grass, road, sky)	205
— of which are waste objects	126
Classes after auto-classification	23 (+ dontknow)

Table 1: Current state. Every image comes from a single camera at a single height, which means the model cannot learn scale or viewpoint invariance from this data.

Class distribution is severely long-tailed:

Class	n	Class	n
dontknow (no object found)	107	glove	4
cigarette_butt	39	candy_wrapper	4
plastic_bottle	17	can	4
plastic_bag	14	pine_cone	3
plastic_wrapper	7	chair	3
textile, plastic_piece, paper	6 each	bottle_cap	3
straw, container, cig_pack	5 each	6 further classes	1 each

2 The resolution limit sets the mission profile

Ground sample distance (GSD) is how much real-world length one pixel covers:

$$\text{GSD} = \frac{2h \tan(\theta/2)}{W}$$

for altitude h , horizontal field of view θ , and image width W pixels.

2.1 Numbers for the DJI Neo 2

The available airframe is a DJI Neo 2: 1/2-inch 12 MP CMOS, **16.5 mm equivalent focal length**, f/2.2, FOV quoted as 119.8°, 4K video at 3840 × 2160 up to 100 fps, 2-axis (tilt/roll) mechanical gimbal, downward infrared plus forward LiDAR obstacle sensing.

This lens is *much wider* than a typical drone camera, which is good for coverage and bad for resolving small objects. Taking the 16.5 mm equivalent gives a horizontal FOV of 95°; reading DJI’s quoted 119.8° as a 16:9 diagonal gives 113°. DJI generally quotes FOV before dewarping, so the truth sits between — the table uses 95° and the text notes the pessimistic case.

Alt.	Swath	GSD	Object size in pixels (longest dimension)			
			Cig. butt	Cap	Wrapper	Bottle
20	43.6	11.36	2.2	2.6	5.3	22
10	21.8	5.68	4.4	5.3	10.6	44
5	10.9	2.84	8.8	10.6	21	88
3	6.5	1.70	14.7	17.6	35	147
2	4.4	1.14	22	26	53	220
1	2.2	0.57	44	53	106	440
0.5	1.1	0.28	88	106	211	880

Table 2: DJI Neo 2, 4K, 95° horizontal FOV. Red entries fall below roughly 20 px, the practical floor at which small-object detectors still work. A cigarette butt is about 8×25 mm.

Inverting this gives the **maximum altitude at which each object class is still detectable** (20 px threshold). The second column is the pessimistic 113° reading — plan against it:

Object	Max. alt. (95°)	Max. alt. (113°)
Cigarette butt	2.2 m	1.6 m
Bottle cap	2.6 m	1.9 m
Wrapper	5.3 m	3.8 m
Plastic bottle	22 m	16 m
Chair / large debris	70 m+	50 m+

These ceilings are roughly **30% lower than for a conventional 24 mm-equivalent drone camera**, purely because of the wide lens. The practical consequences are specific:

- **Cigarette butts require flying at 2 m or below** — realistically 1.5 m to keep margin. That is a low, slow pass demanding good altitude hold; the Neo 2’s downward IR and LiDAR make it feasible, but it is the binding constraint on the whole mission.
- **Even plastic bottles are marginal at 20 m** (16–22 px). The high survey pass should be flown at **10–12 m, not 20 m**, if it is expected to detect anything at all.
- The wide lens partly compensates: at 2 m the swath is 4.4 m, versus 3.0 m for a 24 mm lens. With 20% overlap that is ≈ 2.9 km of track per hectare instead of 3.3 km — still substantial, so the low pass must be targeted rather than exhaustive.

Revised mission profile

The proposed profile — scan at 20 m, then 1–2 m, then buggy — needs one correction: **the 20 m pass cannot be a litter-detection pass**. It should be re-scoped as a *survey* pass, flown lower than planned:

Pass	Altitude	Role
Survey	10–12 m	Large items (bottles, bags, cans, furniture); terrain and surface map; produces the priority regions for the next pass. Not a small-object pass.
Detection	1.5–2 m	Primary small-object detection. Slow, targeted at regions flagged by the survey pass.
Buggy	0.2–1 m	Approach, identification, grasp.

Two Neo-2-specific cautions for the low pass:

- **Downwash.** The Neo 2 is a very light airframe, and at 1.5–2 m its prop wash may physically move cigarette butts, wrappers and dry leaves. This corrupts both detection and the ground truth of a seeded test plot. Test it explicitly before building a protocol on top of it: hover at 2 m over a seeded plot and compare before/after layouts.
- **2-axis gimbal.** Tilt and roll are stabilised; *yaw is not*. Expect residual rotational shake, worse in wind and worse at low altitude in ground effect. Fly the low pass in calm conditions and keep speed down.

A useful lever: the 4K/100 fps mode allows a much shorter effective exposure per frame. For a fixed ground speed this cuts motion blur substantially, which matters more here than frame rate itself — blur is what destroys a 22-pixel object.

3 What needs to be captured

Targets below assume a detector trained per class. Quantities are *annotated object instances*, not images; one image can carry many.

3.1 Altitude and viewpoint coverage

Tier	Height	Angle	Purpose
Buggy-low	0.2 m	0–30° oblique	Final approach, arm alignment
Buggy-high	1 m	nadir + 45°	Buggy search/navigation
Drone-low	2 m	nadir	Primary small-object detection
Drone-low	5 m	nadir + 30°	Transition scale
Drone-high	10 m	nadir	Large-object detection
Drone-high	20 m+	nadir	Survey, terrain prioritisation

The single most valuable capture method is the *altitude ladder*: place a known object, then photograph the identical scene at 0.2, 1, 2, 5, 10 and 20 m without moving anything. This yields exact cross-scale correspondence, lets you measure empirically where each class stops being detectable, and supports scale-augmentation training. Repeat per surface type. Roughly 50 ladders would transform the dataset’s usefulness.

Angles matter because the buggy sees objects obliquely while the drone sees them from directly above — these look nothing alike. Every class needs both.

3.2 Occlusion

Currently unrepresented as a labelled axis. Capture each class at four levels, and **record the occlusion fraction in the annotation** so it can be evaluated separately:

Level	Definition
None	Fully visible against surface
Light	< 25% hidden; grass blades crossing the object
Partial	25–60% hidden; in grass tufts, under leaf edges
Heavy	> 60% hidden; only a fragment protrudes

Occluders to cover explicitly: tall grass, cut grass, dead leaves, loose soil, gravel, twigs/pine needles, puddles, and *other litter* (overlapping objects, which the pipeline will meet in dumping hot-spots).

3.3 Surface, lighting, weather

Backgrounds already present are gravel, soil and grass. Missing and needed: asphalt, concrete/paving, sand, wood chip, leaf litter, and short mown lawn. Surface matters more than usual here because low-contrast pairings are the hard cases — a beige butt on gravel, a clear bottle on wet asphalt.

Lighting in the current set is 230/244 bright sunlight. Required additions: overcast, deep shadow, dappled shade under trees, golden hour with long shadows, and dawn/dusk. Add wet-surface and post-rain conditions. If night operation is intended, capture under drone illumination.

3.4 Negatives

The 107 dontknow images are an asset, not waste — they are **hard negatives**: real ground with no litter. Keep them as explicitly labelled empty frames. Add deliberate confusers: pine cones, stones, leaves, bark, dog waste, twigs, mushrooms, and flowers, all of which resemble litter at low pixel counts. Target 20–30% of the final set as negatives.

3.5 Volume targets

Class tier	Instances (minimum)	Instances (comfortable)
Priority (cigarette butt, bottle, can, wrapper, bag)	1,000	3,000
Secondary (cup, cap, straw, glove, textile, paper)	500	1,500
Rare / large (chair, crate, furniture)	200	500
Hard negatives (empty ground, confusers)	2,000	5,000

Each priority-class instance count should be spread across the altitude tiers, not concentrated at one height. A practical rule: no more than 40% of a class’s instances from any single altitude tier.

4 Annotation requirements

The existing polygon workflow is sound and should continue — polygons are worth the extra effort here because they support segmentation, and grasp planning for the arm needs object outline, not just a box.

Changes needed:

1. **Fix the label vocabulary.** Current labels mix Danish and English (*serviet*, *krukker*, *grankogler*) and contain typos and truncations (*dirty_clot*, *dirt_r*, *bottle_*, *gra*, 1, 2). Freeze a controlled vocabulary and validate against it at annotation time.
2. **Separate background from object classes.** 205 of 331 polygons label terrain (rock, grass, road, sky). These are useful for a segmentation task but must not be mixed into the detector’s object classes.
3. **Record per-instance metadata:** altitude, camera angle, occlusion level, surface, lighting. Without these the evaluation in Section 5 cannot be broken down, and you will not know *where* the model fails.
4. **Add a scale reference** to capture sessions — a coloured marker of known size in frame lets you verify GSD and recover real object dimensions, which the arm needs for grasp sizing.
5. **Verify a sample by hand.** Spot-checks during this analysis found manual labels that disagree with the imagery (one *paper* polygon contains a drinking straw; *dirty_cloth* is most likely automotive carpet). Budget a second-pass review.

5 Test videos for offline evaluation

Video is the right evaluation medium because the deployed system sees motion, motion blur and repeated views of the same object — and because a video lets you measure *tracking consistency*, which single images cannot.

Neo 2 recording settings

Setting	Value and reason
Resolution	4K 3840 × 2160. Never 1080p — halving resolution doubles the minimum detectable object size.
Frame rate	60 fps normally; 100 fps for all moving takes. The shorter exposure is what preserves small objects, not the frame count.
Bitrate/codec	Highest available (80 Mbps). Compression artefacts and 20-pixel objects interact badly.
Colour	Standard profile, not D-Log, unless you will apply the identical transform at inference.
Gimbal	Nadir (−90°) for survey and sweep takes; −45° for the oblique set.
Stabilisation	Record at least a third of takes with EIS off, so the model sees the real vibration signature it will meet in deployment.
Flight	Calm conditions for low passes. Log altitude, speed and GPS for every take.

Before each take, place a known object set and photograph a ground-truth layout map from directly above, so every object in the scene has a known identity and position. Keep takes to 60–90 s. Fly the same plot across all conditions so that comparisons isolate one variable at a time.

Required takes

Take	Profile	What it measures
Altitude descent	Hover, descend 15 m → 0.5 m over a fixed seeded plot, pausing 5 s at 15, 10, 5, 3, 2, 1, 0.5 m	Fly this one first. It gives the empirical detection threshold per class. The altitude at which each object first appears is the single most useful number you can collect, and it validates or corrects Table 2 against your actual airframe.
High survey	10–12 m, 3 m/s, straight transects	Large-object recall and false-positive rate over open ground. Flown at 10–12 m rather than 20 m per Section 2.
Low sweep	1.5–2 m, 1 m/s, parallel tracks with 20% overlap (4.4 m swath)	Primary small-object detection performance; also validates that the swath/overlap plan actually achieves full coverage.
Speed sweep	2 m at 0.5, 1, 2, 4 m/s; repeat at 60 and 100 fps	Where motion blur breaks detection — sets the operational speed limit, and quantifies what the 100 fps mode buys you.

Take	Profile	What it measures
Occlusion plot	2 m over a plot seeded at all four occlusion levels	Recall as a function of occlusion; the number to quote when scoping expectations.
Surface transition	2 m crossing grass→gravel→asphalt	Background-shift robustness within a single continuous clip.
Lighting series	Same plot at dawn, noon, overcast, dusk, wet	Illumination sensitivity with all else held constant.
Buggy approach	1 m→0.2 m driving toward a target	The handoff: does the object stay tracked as it grows and the angle flattens?
Buggy patrol	0.5 m, continuous drive through a seeded area	End-to-end ground-stage recall and duplicate-detection behaviour.
Grasp approach	0.2 m static, arm entering frame	Whether detection survives occlusion <i>by the robot's own arm</i> — a failure mode that is easy to forget and fatal in practice.
Confuser plot	2 m and 0.5 m over pine cones, stones, leaves only	False-positive rate on a scene containing zero litter.
Downwash check	Hover 60 s at 2 m, then 1 m, over a seeded plot	Whether the aircraft's own prop wash displaces the targets. Not a detection test — a validity test for every other low take.
Empty control	Each altitude over verified-clean ground	Baseline false-positive rate. Any detection here is an error, which makes this the cheapest take to score.

Seeded plots. Lay out a 5×5 m plot with a known inventory — e.g. 20 cigarette butts, 5 bottles, 5 cans, 10 wrappers, 3 bags — at recorded positions. Because the count is known exactly, recall can be computed without annotating every frame. Build three plots: sparse (realistic), dense (hot-spot), and confuser-heavy.

Scoring the videos

Annotate every N -th frame (10–30) rather than all frames, and report precision/recall broken down by altitude, occlusion level, surface and class — aggregate numbers will hide exactly the failures that matter. Track two video-specific metrics: **re-detection stability** (fraction of frames in which an object, once seen, stays detected) and **time-to-first-detection** during descent.

Keep every test video strictly out of training. Given GPS is recorded, split train/test **by location**, not randomly — random splits across frames of the same site inflate scores badly.

6 Cheap baselines: Google, OpenAI and Hugging Face models

Before training anything, it is worth knowing what off-the-shelf models already achieve on this data — both as a performance floor and as an annotation accelerator. This can be done for a few dollars if the work is organised correctly, and it is easy to spend a hundred times more than necessary by organising it badly.

6.1 What these models are and are not for

None of these are the deployed detector. The drone and buggy need an onboard model with millisecond latency and no network. Cloud vision models serve three other purposes:

1. **A baseline floor.** If a zero-shot model already finds 80% of your bottles, a custom detector must beat that to justify itself.
2. **Auto-labelling.** Pre-annotation that a human corrects is several times faster than annotating from scratch — the main lever on the annotation cost of a 10^3 -instance dataset.
3. **Adjudication.** A second opinion on ambiguous frames.

A crucial distinction: **general vision-language models (GPT, Gemini) are not detectors.** They describe an image; they do not reliably produce boxes for 20-pixel objects. A trial run of GPT-4o over this image set at low detail reported “no waste item” for 107 of 244 images, including photographs containing a plainly visible rusty metal bar. **Open-vocabulary detectors are the right comparison** — they take a text prompt and return boxes, which is exactly the target task.

6.2 Hugging Face: the free tier that matters

Models run locally cost nothing per image, which makes them the correct default for anything involving video, where frame counts run to the tens of thousands. All of the following run on a recent laptop:

Model	Task	Use here
OWLv2	Open-vocab detection	Zero-shot boxes from prompts such as “cigarette butt”. Primary base-line.
Grounding DINO	Open-vocab detection	Stronger phrase grounding; cross-check against OWLv2.
YOLO-World	Open-vocab detection	Real-time; closest in spirit to what will run onboard.
SAM 2	Segmentation	Turns a box into a polygon — converts cheap box labels into the polygon masks this project already uses.
Florence-2	Caption + grounding	Small, fast, dense captioning for bulk triage.
Qwen2.5-VL	VLM	Local stand-in for GPT/Gemini-class description.

The highest-value combination is **Grounding DINO or OWLv2 for boxes, then SAM 2 to convert each box to a polygon**, giving pre-annotations in the project’s existing format at zero marginal cost. Run this over every frame of every test video; it is free, so there is no reason to sample.

6.3 Cost mechanics of the paid APIs

Measured token counts for this dataset (1024px images, 250-token prompt, JSON output) give the following for all 244 images:

Model	In/img	Out/img	Standard	Batch (−50%)
gpt-5.4-mini	1136	123	\$0.34	\$0.17
gpt-4o (detail low)	300	101	\$0.43	\$0.21
gpt-4o (detail high)	980	99	\$0.84	\$0.42
gpt-5.4	1136	131	\$1.17	\$0.59
Gemini Flash	1136	130	\$0.54	\$0.27
gpt-5.6-sol	1136	246	\$3.19	\$1.59

Six levers, in order of effect:

1. **Batch APIs halve the price.** Both OpenAI and Google discount asynchronous batch jobs by 50%. Offline evaluation has no latency requirement, so there is never a reason to pay standard rates for it.
2. **Google’s free tier is genuinely free.** Gemini Flash models are available at no cost in AI Studio (order 1,500 requests/day). That covers the entire 244-image set daily for nothing. **Caveat: free-tier content may be used to improve Google’s products.** If the dataset is to remain proprietary, use the paid tier — at \$0.27 batched, that is a cheap way to keep it private.
3. **Downscale before upload.** Images are 12 MP; models resample them anyway. Sending 1024px costs the same as sending 4032px in tokens but a fraction of the bandwidth, and above the model’s internal tile limit the extra pixels are simply discarded.
4. **Know which models honour detail:** low. Measured on this data: GPT-4o honours it (300 vs. 980 tokens); **GPT-5.4 ignores it entirely** (1136 either way); GPT-5.6-sol honours it (445 vs. 1136). Requesting low detail on a model that ignores it saves nothing and loses accuracy on the ones that do not.
5. **Cascade.** Run the free local model on everything, a cheap API model on what it flags, and an expensive model only on disagreements. In practice the expensive tier sees 5–10% of frames.
6. **Cache every result.** Store raw responses keyed by image hash and never re-run the same evaluation. Re-running a finished job is the most common way this budget is wasted.

6.4 Video without paying for video

Do not send raw video to a paid API. Gemini bills video at roughly 5,800 tokens per second of 720p footage. A single 90s clip is about 521,000 tokens — \$0.78 on Flash, \$0.39 batched — and the full 13-take suite comes to roughly \$5 per evaluation pass on the cheapest model, more on anything larger. That is affordable once and ruinous if iterated, and iteration is the entire point of an offline test set.

The alternative costs almost nothing and gives strictly more control:

1. Extract frames locally with `ffmpeg` at 1–2 fps.
2. Run the local Hugging Face detectors over *every* frame — free, and the only way to measure per-frame tracking stability anyway.
3. Draw a **stratified sample of 200 frames** spanning altitude, occlusion, surface and lighting, and send only those to the paid APIs.

At 200 frames the paid comparison costs \$0.14 (gpt-5.4-mini batched), \$0.27 (Gemini Flash batched) or \$0.48 (gpt-5.4 batched) *per model*. Comparing all three across the whole test suite is under a dollar — against roughly \$5 per pass for native video on one model alone.

Native video input is still worth one deliberate run, for the questions frames cannot answer: temporal consistency, and whether a model tracks an object through occlusion. Budget it as a single experiment, not a routine.

6.5 Suggested comparison protocol

Stage	Model	Cost
All frames, all videos	OWLv2 / Grounding DINO / SAM 2, local	\$0
All 244 stills	Gemini Flash, free tier	\$0
244 stills, private	Gemini Flash batch	\$0.27
200-frame stratified sample	gpt-5.4-mini batch	\$0.14
200-frame sample, second opinion	gpt-5.4 batch	\$0.48
Disagreements only (≈ 20 frames)	gpt-5.6-sol	$< \$0.15$
One native-video run	Gemini Flash batch, 1 take	\$0.39
Total per full evaluation pass		$\approx \\$1.45$

Score every model on the same held-out frames with the same metric, broken down by altitude and occlusion. The interesting output is not a leaderboard but a **map of where each model fails**, which tells you which conditions the training set most needs to cover.

7 Suggested order of work

1. Freeze the label vocabulary; fix the existing typo/language labels.
2. Run the free local baseline (Section 6) over the existing 244 images. It costs nothing, and if an off-the-shelf detector already handles a class well, that class needs less new data than the others.
3. Fly one altitude-descent test video. It costs an afternoon and tells you the real per-class detection ceiling before you invest in bulk capture.
4. Decide the mission profile from that measurement (Section 2 gives the prediction; the flight gives the truth).
5. Bulk-capture the altitude ladders across surfaces and lighting.
6. Annotate with the extended per-instance metadata.
7. Record the full test-video suite; keep it sealed from training.

Figures in Sections 1–2 are computed from the current repository contents and from the stated 4K/24 mm-equivalent camera assumption; substitute the real lens parameters when the airframe is chosen, as GSD scales linearly with focal length.